

Importing Libraries:

```
In [4]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import (confusion_matrix, classification_report, accu
```

Loading Data Files

```
In [12]: odi_df = pd.read_csv('virat_kohli_odi_innings_data.csv')
t20_df = pd.read_csv('virat_kohli_t20i_innings_data.csv')
test_df = pd.read_csv('virat_kohli_test_innings_data.csv')

print("ODI shape :", odi_df.shape)
print("T20 shape :", t20_df.shape)
print("TEST shape:", test_df.shape)
```

```
ODI shape : (262, 13)
T20 shape : (114, 13)
TEST shape: (180, 13)
```

```
In [13]: odi_df.head()
```

```
Out[13]:
```

	Runs	Mins	BF	4s	6s	SR	Pos	Dismissal	Inns	Opposition	Ground	
0	12	33	22	1	0	54.54	2	lbw	1	v Sri Lanka	Dambulla	18,
1	37	82	67	6	0	55.22	2	caught	2	v Sri Lanka	Dambulla	20,
2	25	40	38	4	0	65.78	1	run out	1	v Sri Lanka	Colombo (RPS)	24,
3	54	87	66	7	0	81.81	1	bowled	1	v Sri Lanka	Colombo (RPS)	27,
4	31	45	46	3	1	67.39	1	lbw	2	v Sri Lanka	Colombo (RPS)	29,

```
In [14]: t20_df.head()
```

Out [14]:

	Runs	Mins	BF	4s	6s	SR	Pos	Dismissal	Inns	Opposition	Ground
0	26	30	21	3	1	123.8	5	not out	2	^v Zimbabwe	Harare
1	DNB	-	-	-	-	-	-	-	2	^v Zimbabwe	Harare
2	28	28	19	5	0	147.36	1	bowled	1	^v South Africa	Durban
3	14	15	12	2	0	116.66	3	caught	1	^v West Indies	Port of Spain
4	4	5	5	0	0	80	4	caught	1	^v England	Manchester

In [15]: test_df.head()

Out [15]:

	Runs	Mins	BF	4s	6s	SR	Pos	Dismissal	Inns	Opposition	Ground
0	4	13	10	1	0	40	5	caught	1	^v West Indies	Kingston
1	15	72	54	2	0	27.77	5	caught	3	^v West Indies	Kingston
2	0	1	2	0	0	0	5	caught	1	^v West Indies	Bridgetown
3	27	118	107	1	1	25.23	5	caught	3	^v West Indies	Bridgetown
4	30	62	53	2	0	56.6	5	caught	2	^v West Indies	Roseau

Converting 'Runs' and 'SR' to numeric, handling 'DNB' and 'TDNB' values and
Cleaning the data:

```
In [38]: # Converting 'Runs' and 'SR' to numeric, handling 'DNB' and 'TDNB' values
odi_df['Runs'] = pd.to_numeric(odi_df['Runs'], errors='coerce')
t20_df['Runs'] = pd.to_numeric(t20_df['Runs'], errors='coerce')
test_df['Runs'] = pd.to_numeric(test_df['Runs'], errors='coerce')

odi_df['SR'] = pd.to_numeric(odi_df['SR'], errors='coerce')
t20_df['SR'] = pd.to_numeric(t20_df['SR'], errors='coerce')
test_df['SR'] = pd.to_numeric(test_df['SR'], errors='coerce')

odi_df.dropna(subset=['Runs', 'SR'], inplace=True)
```

```
t20_df.dropna(subset=['Runs', 'SR'], inplace=True)
test_df.dropna(subset=['Runs', 'SR'], inplace=True)

print("\nAfter cleaning:")
print("ODI shape :", odi_df.shape)
print("T20 shape :", t20_df.shape)
print("TEST shape:", test_df.shape)
```

After cleaning:

ODI shape : (252, 13)

T20 shape : (106, 13)

TEST shape: (173, 13)

Concatenating runs, strike rate and grounds and storing into separate list

```
In [18]: Runs = np.concatenate((odi_df['Runs'].values, t20_df['Runs'].values, test_df['Runs'].values),
SR = np.concatenate((odi_df['SR'].values, t20_df['SR'].values, test_df['SR'].values),
Ground = np.concatenate((odi_df['Ground'].values, t20_df['Ground'].values, test_df['Ground'].values),
```

Step 4: Create match_type labels

```
In [39]: match_type = np.concatenate((
    np.repeat("ODI", len(odi_df['Runs'])),
    np.repeat("T20", len(t20_df['Runs'])),
    np.repeat("TEST", len(test_df['Runs']))
))
```

Step 5: Create data_cricketer DataFrame

```
In [40]: data_cricketer = pd.DataFrame({'Runs':Runs, 'SR':SR, 'match_type': match_type})
print("\nFinal dataset shape:", data_cricketer.shape)
print(data_cricketer.head())
```

Final dataset shape: (531, 4)

	Runs	SR	match_type	Ground
0	12.0	54.54	ODI	Dambulla
1	37.0	55.22	ODI	Dambulla
2	25.0	65.78	ODI	Colombo (RPS)
3	54.0	81.81	ODI	Colombo (RPS)
4	31.0	67.39	ODI	Colombo (RPS)

Step 6 and 7: Define features X and target Y and Train/Test split:

```
In [23]: X = data_cricketer[['Runs', 'SR']].values
Y = data_cricketer['match_type'].values
```

```
In [24]: x_train, x_test, y_train, y_test = train_test_split(
    X, Y, test_size=0.3, random_state=0
)

print(f"\nTraining samples : {x_train.shape[0]}")
print(f"Testing samples : {x_test.shape[0]}")
```

Training samples : 371

Testing samples : 160

Step 8: Create SVC with Polynomial kernel degree=6, fit & predict

```
In [25]: svm_model = SVC(kernel='poly', degree=6, random_state=0)
svm_model.fit(x_train, y_train)
y_pred = svm_model.predict(x_test)
```

Step 9: Compute confusion matrix, accuracy and balanced accuracy

```
In [26]: cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
bal_acc = balanced_accuracy_score(y_test, y_pred)

print("\n--- Confusion Matrix ---")
print(cm)
print(f"\nAccuracy : {acc:.4f}")
print(f"Balanced Accuracy : {bal_acc:.4f}")
```

--- Confusion Matrix ---

```
[[48  2 23]
 [13 17  3]
 [ 1  0 53]]
```

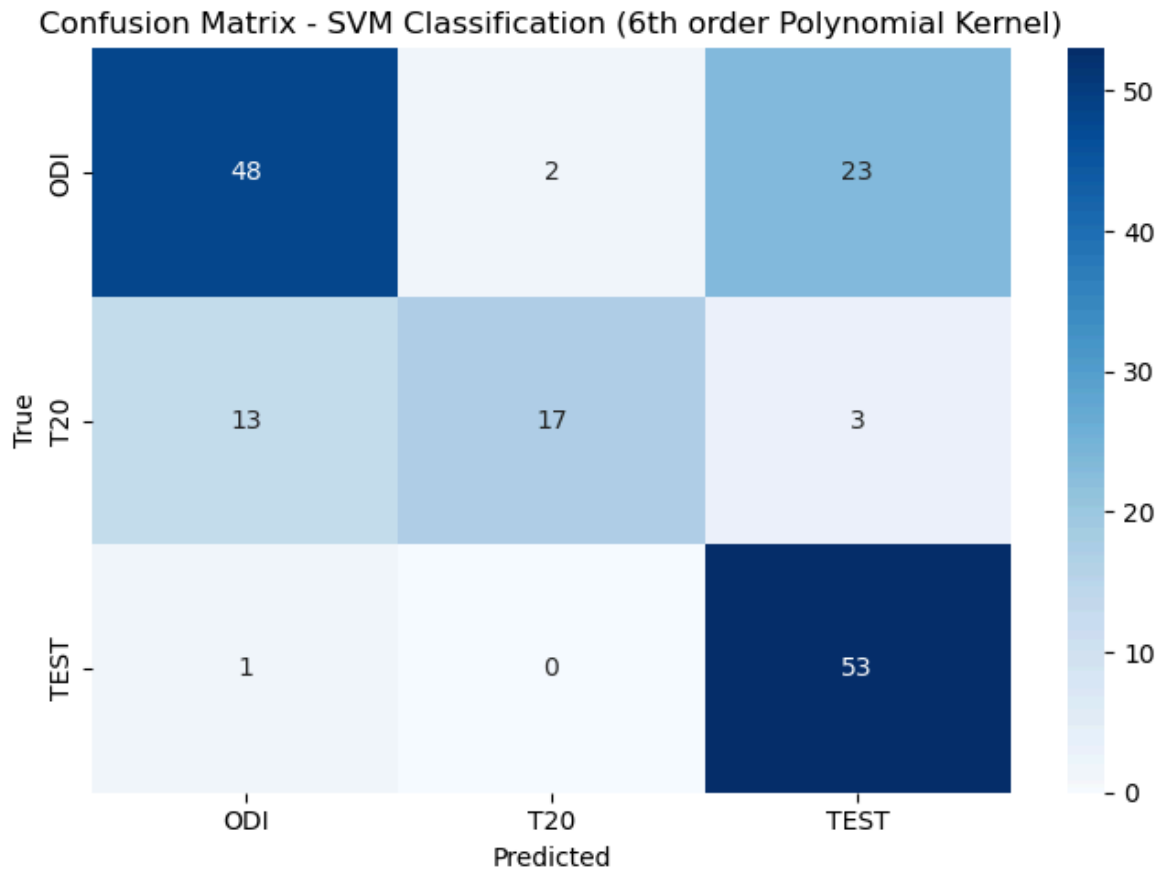
Accuracy : 0.7375

Balanced Accuracy : 0.7181

Step 10: Plot Confusion Matrix:

```
In [30]: labels = sorted(data_cricketer['match_type'].unique())

plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=labels, yti
plt.title('Confusion Matrix - SVM Classification (6th order Polynomial Ke
plt.xlabel('Predicted')
plt.ylabel('True')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150)
plt.show()
```



Step 11: Classification report

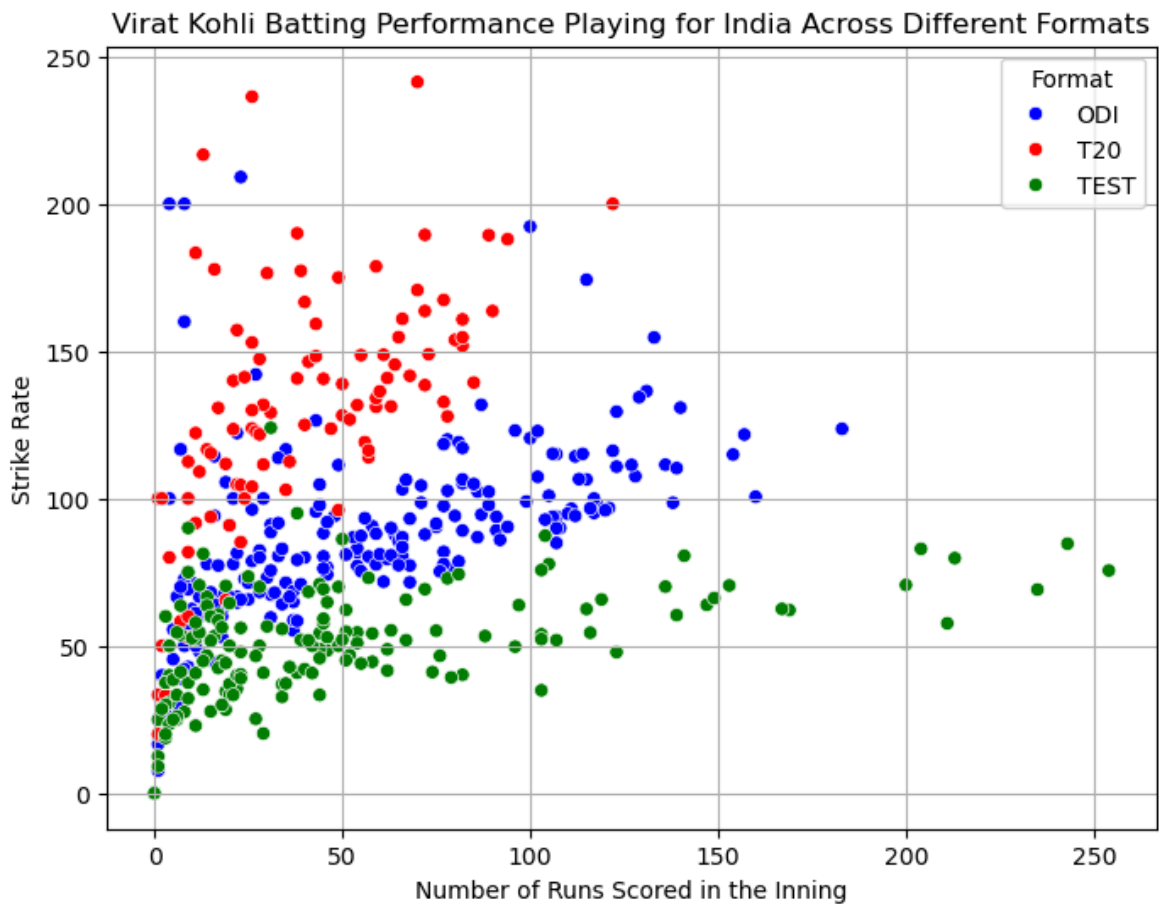
```
In [31]: print("\n--- Classification Report ---")
print(classification_report(y_test, y_pred, target_names=labels))
```

```
--- Classification Report ---
```

	precision	recall	f1-score	support
ODI	0.77	0.66	0.71	73
T20	0.89	0.52	0.65	33
TEST	0.67	0.98	0.80	54
accuracy			0.74	160
macro avg	0.78	0.72	0.72	160
weighted avg	0.76	0.74	0.73	160

Step 12: Scatter plot - Runs vs Strike

```
In [45]: plt.figure(figsize=(8, 6))
sns.scatterplot(x=data_cricketer['Runs'], y=data_cricketer['SR'], hue=data_cricketer['Format'])
plt.xlabel('Number of Runs Scored in the Inning')
plt.ylabel('Strike Rate')
plt.title('Virat Kohli Batting Performance Playing for India Across Different Formats')
plt.legend(title='Format')
plt.grid()
plt.show()
```



Report:

1. What are Support Vectors in SVM? How do they help in making SVM less prone to outliers?

Support Vectors are the data points closest to the decision boundary and are used to define the optimal hyperplane. Since SVM relies only on these critical points, distant data points and outliers have minimal effect on the model, making it more robust.

2. What is the difference between a hard margin and a soft margin in SVM?

Hard margin SVM does not allow any misclassification and assumes perfectly separable data. Soft margin SVM allows some misclassification, making it more flexible and better suited for noisy or real-world data.

3. How does SVM deal with non-linear classification boundaries?

SVM uses kernel functions to project data into a higher-dimensional space where it becomes linearly separable. This allows SVM to model complex non-linear decision boundaries effectively.

4. What is the role of gamma in SVM? How does the value of gamma impact underfitting/overfitting?

Gamma controls the influence of individual data points on the decision boundary. Low gamma leads to a smoother boundary and possible underfitting, while high gamma creates a complex boundary and may cause overfitting.

5. How are SVMs used for multi-class classification problems?

SVM handles multi-class problems using One-vs-Rest or One-vs-One strategies, where multiple binary classifiers are trained and combined to make the final prediction.